Optimizing Colors in Fixed-Geometry 3D Gaussian Splatting: First-Order and Krylov Methods

Anonymous Author(s)
Affiliation
Address
email

Abstract

Optimizing all parameters of a 3D Gaussian scene confounds optimizer behavior with changing geometry and visibility. This study isolates appearance: after a common geometry preparation stage, positions, anisotropic covariances, opacities, cameras and the Gaussian population are frozen, and only view-independent RGB coefficients are optimized. With unclipped alpha compositing and a squared photometric objective, this is an exactly linear least-squares problem with a strictly convex ridge regularizer. We implement gradient descent, strongly convex Nesterov acceleration, Adam, conjugate gradients (CG), and diagonally preconditioned CG (PCG) using the existing CUDA rasterizer as a matrix-free operator. A nonnegative-compositing argument supplies both a conservative Lipschitz bound and a positive diagonal preconditioner. Experiments on the Train and Truck scenes of Tanks and Temples compare equal-iteration optimization, stationarity, elapsed solver time and held-out image quality under three matched color perturbations. All reported measurements are generated from saved runs; geometry checksums, view lists, raw traces and executable code accompany the report. The conclusions concern this fixed-geometry subproblem rather than full 3DGS reconstruction.

1 Introduction

Multi-view scene reconstruction combines geometric estimation with appearance fitting. In 3D Gaussian Splatting (3DGS), each primitive has a center, covariance, opacity and appearance coefficients. Joint training also creates and removes primitives. An optimizer comparison conducted during this process is difficult to interpret: one algorithm may see a different number of variables, a different visibility pattern, or a different geometric initialization. A smaller, controlled problem can expose the relationship between scene rendering and the optimization concepts studied in COMP6704.

The question here is: *when geometry and visibility are fixed, which optimization methods most effectively fit Gaussian colors to multiple calibrated images?* This remains a real image-fitting task involving many color variables, overlapping translucent primitives and unequal observation frequencies. It also has useful mathematical structure. Fixed alpha-compositing weights make the rendered image linear in RGB coefficients. Squared error and positive ridge regularization then yield a symmetric positive-definite normal equation, making both first-order and Krylov methods applicable.

The contributions are a reproducible five-method comparison, a derivation connecting the renderer to a convex quadratic, and a simple diagonal preconditioner justified by nonnegative blending weights. The preconditioner is an adaptation of standard numerical linear algebra to this appearance model; it is not presented as a new general optimization algorithm. The design deliberately limits scope to diffuse RGB appearance and short, common geometry preparation. This makes the experiments practical while preserving a precise optimization question.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

2 Background and related work

Scene representation. Kerbl et al. (2023) introduced anisotropic 3D Gaussians, differentiable visibility-aware rasterization and adaptive density control for fast radiance-field rendering. Their full optimization is nonconvex because geometry, opacity and the primitive population change. This report retains the renderer and Gaussian representation but freezes those variables. Degree-zero spherical harmonics correspond to a view-independent RGB value per Gaussian. Higher-degree appearance, densification and exposure optimization are outside the comparison.

First-order optimization. Gradient descent relates convergence to the Hessian spectrum and motivates studying conditioning (Boyd and Vandenberghe, 2004). Nesterov acceleration uses extrapolation to improve worst-case convergence for smooth convex problems (Nesterov, 1983). Adam uses bias-corrected first and second moment estimates (Kingma and Ba, 2015). Its popularity in scene optimization makes it a relevant practical baseline, but popularity does not imply guaranteed convergence for every convex problem; Reddi et al. (2018) identify failures of the original convergence argument. We therefore measure Adam's finite-budget behavior rather than claim a universal rate for it.

Krylov and recent 3DGS optimization. CG solves symmetric positive-definite linear systems using matrix-vector products (Hestenes and Stiefel, 1952). Preconditioning changes the effective spectrum without changing the exact minimizer. Recent work also uses curvature for full Gaussian optimization. 3DGS-LM applies a tailored Levenberg–Marquardt method (Höllein et al., 2025); matrix-free second-order optimization with residual sampling exploits sparse Gaussian-to-pixel dependencies (Pehlivan et al., 2026); and 3DGS^2-TR studies diagonal curvature and trust regions (Hsiao et al., 2026). These methods address a broader nonlinear problem. Our CG system is instead the exact, fixed appearance Hessian, not a local Gauss–Newton approximation. Their published speedups are not imported as experimental results or used as directly comparable baselines.

Data and evaluation. Tanks and Temples is a benchmark for reconstruction from real multi-view imagery (Knapitsch et al., 2017). We use the locally available COLMAP-preprocessed Train and Truck scenes. Our measurements concern photometric appearance fitting and image generalization, not the benchmark's laser-scan geometry ranking. This distinction prevents PSNR improvements from being interpreted as improvements in reconstructed surface accuracy.

3 Problem formulation

3.1 From alpha compositing to a linear operator

For a pixel p in view v, let $\mathcal{O}_{vp}$ be the ordered list of contributing Gaussians. With frozen geometry and opacity, the projected opacity α_{vpi} and transmittance are fixed. Define

$$w_{vpi} = \alpha_{vpi} \prod_{j \prec i} (1 - \alpha_{vpj}), \qquad I_{vp}(c) = \sum_{i \in \mathcal{O}_{vp}} w_{vpi} c_i. \tag{1}$$

The background is black. Frustum culling, opacity thresholds and early termination depend on geometry and opacity, not on the supplied RGB values. Consequently these implementation choices still produce a fixed operator. Stack all fitted image channels into Ac, where A includes three identical channel blocks. The weights are nonnegative. No sigmoid, RGB clamp, learned exposure, SSIM or absolute-value term is included in the optimized objective.

This qualification matters in the actual installation: its high-level renderer clamps the final image to $[0, 1]$. Using that function would destroy global linearity. Our independent implementation calls the underlying CUDA rasterizer with precomputed RGB colors and takes its raw output. Clipping is applied only for separately labelled display metrics and figures.

3.2 A strictly convex appearance objective

Let c^0 be the RGB coefficients after common geometry preparation, let $x = c - c^0$ be a color correction, and define $b = y - Ac^0$ for the fitted target images y. With m observed scalar image

values, minimize

$$F(x) = \frac{\|Ax - b\|_2^2 + \lambda\|x\|_2^2}{2m}, \qquad x \in \mathbb{R}^{3N}, \quad \lambda > 0. \tag{2}$$

The ridge anchors poorly observed coefficients to their original colors. It is centered at the common preparation colors, not at the random starting iterate. For implementation convenience we optimize mF, whose stationary point solves

$$Hx = q, \qquad H = A^T A + \lambda I, \qquad q = A^T b. \tag{3}$$

For every nonzero z, $z^T H z = \|Az\|^2 + \lambda\|z\|^2 > 0$. Thus H is symmetric positive definite and the minimizer is unique, even if some Gaussians are never visible in fitted views. For such variables the optimum correction is zero. Unconstrained colors can leave the physical RGB range; this is an explicit modeling tradeoff, not an omitted constraint.

3.3 A usable bound and diagonal preconditioner

Materializing A or H is unnecessary. The renderer computes Az, and its color backward pass computes $A^T u$. Let $d = H\mathbf{1}$ and $D = \mathrm{diag}(d)$. Since every entry of H is nonnegative,

$$z^T(D - H)z = \frac{1}{2}\sum_{i,j} H_{ij}(z_i - z_j)^2 \geq 0. \tag{4}$$

Therefore $H \preceq D \preceq LI$ for $L = \max_i d_i$, while $H \succeq \lambda I$. One matrix-free product obtains d. We use L for safe scalar steps and D as a PCG preconditioner. It is a row-sum majorizer, *not* the exact Jacobi diagonal $\mathrm{diag}(H)$. The bound can be conservative when observation coverage is very unequal. It guarantees an upper spectral bound in exact arithmetic; it does not guarantee that this particular preconditioner always improves finite-precision CG.

4 Solution methods and implementation

All methods use the same quadratic, initialization and maximum number of updates. GD takes

$$x_{k+1} = x_k - L^{-1}(Hx_k - q). \tag{5}$$

For Nesterov acceleration, set $x_{-1} = x_0$, $\beta = (\sqrt{L} - \sqrt{\lambda})/(\sqrt{L} + \sqrt{\lambda})$, and use

$$u_k = x_k + \beta(x_k - x_{k-1}), \qquad x_{k+1} = u_k - L^{-1}(Hu_k - q). \tag{6}$$

For the unnormalized objective, the condition number is at most $\kappa = L/\lambda$. GD has a linear worst-case rate governed by $1 - 1/\kappa$; accelerated gradient has the familiar $1 - 1/\sqrt{\kappa}$ dependence, up to the usual objective/potential constants. These are conservative bounds, not measured iteration predictions. Extrapolation can make Nesterov's individual objective values nonmonotone.

Adam maintains $s_k = 0.9s_{k-1} + 0.1g_k$ and $v_k = 0.999v_{k-1} + 0.001g_k^2$, applies the standard bias corrections, and updates x using $\eta\widehat{s}_k/(\sqrt{\widehat{v}_k} + 10^{-8})$. Here $g_k = Hx_k - q$. Its step size is selected from $\{0.003, 0.01, 0.03\}$ using a 40-update pilot on appearance-validation images. The pilot costs and chosen rates are saved. The main optimizer comparison is full batch; "Adam" therefore does not imply stochastic minibatching in this experiment.

For CG, initialize $r = q - Hx$ and $p = r$. An update uses $\alpha = (r^T r)/(p^T Hp)$, $x \leftarrow x + \alpha p$, $r_{\mathrm{new}} \leftarrow r - \alpha Hp$ and the usual conjugate-direction recurrence. PCG replaces residuals in the inner products by $z = D^{-1}r$: $\alpha = (r^T z)/(p^T Hp)$ and $\beta = (r_{\mathrm{new}}^T z_{\mathrm{new}})/(r^T z)$. The three RGB blocks are treated as one vector. Both methods recompute the true residual and restart the direction every 25 updates to limit float32 recurrence drift. Thus the runs are periodically restarted CG/PCG, and exact-arithmetic termination within the dimension is not claimed for this implementation.

Each update requires one H product except residual replacement, which requires a second product. Let C_A denote the cost of one full fitted-view forward and backward sweep. The leading per-update cost is $O(C_A + N)$ for all five methods; several vector arrays are needed for momentum and Krylov states. The sparse rasterizer avoids dense $O(mN)$ matrices. Cached backward graphs still retain view-specific rasterization buffers, so memory is not simply $O(N)$ overall. Preconditioner construction and common setup are measured separately from solver updates.

Correctness checks. Before each dataset run, numerical checks test linear combinations of rendered colors, the inner-product identity $\langle Au, t\rangle = \langle u, A^T t\rangle$, symmetry of H, and centered finite differences of Eq. 2. An independent small positive-definite problem compares every solver with a dense direct solution. Geometry checkpoint hashes are checked again after appearance optimization. These tests target the central modeling assumptions rather than image quality alone.

5 Experimental design

Preparation and split. Both scenes start from the provided COLMAP data. The source dataset and existing experiments remain outside the independent output directory. Images are ordered by name; every eighth image is held out, matching the installed 3DGS evaluation convention. A fresh, degree-zero model is prepared for 3,000 updates at width 512 using only the remaining image targets. Preparation uses the installed default Adam and $0.8L_1 + 0.2(1 - \mathrm{SSIM})$ loss; this differs from the subsequent color quadratic. Densification stops at update 2,000, and opacity reset is deferred beyond this run. All five appearance methods then share the same saved geometry. COLMAP cameras and sparse initialization originate from the supplied reconstruction, which may itself use all input images; the image-target holdout is not a claim of independently reconstructed test geometry.

For color fitting, eight uniformly spaced views are selected from the geometry-training pool after removing every eleventh image as appearance validation. Main images have width 256 with preserved aspect ratio. Validation targets were used during common geometry preparation, but never enter the color quadratic; they select Adam's appearance step size. The test images enter neither geometry-target training nor color fitting or hyperparameter selection. All split lists are saved explicitly. This nested protocol tests the appearance subproblem, not end-to-end model selection from unseen geometry.

Matched initializations and budgets. For seeds 0, 1 and 2, every method starts from $x_0 \sim \mathcal{N}(0, 0.1^2 I)$, using identical samples within a seed. This is a controlled color-recovery and refinement task around a shared reconstructed scene. The principal budget is 100 updates with $\lambda = 0.01$. Method order is cyclically shifted across seeds to reduce a fixed ordering effect. The seeds vary color perturbations, not geometry training, camera splits or scene identity. They must not be interpreted as three independent scene reconstructions.

Metrics and timing. We report the normalized objective F, the true residual ratio $\|Hx_k - q\|/\|Hx_0 - q\|$, solver seconds, and arithmetic mean per-view PSNR on all test views. Raw PSNR uses the unclipped model; clipped PSNR and SSIM characterize display quality separately. The plotted residual is recomputed from the actual iterate rather than copied from CG's recurrence. CUDA is synchronized around timed updates. Histories, evaluation, common setup and Adam pilots are excluded from solver time, while the initial gradient and periodic residual replacements are included. Thus solver time is not end-to-end training time. The server is shared, and timing is descriptive rather than a dedicated-machine speed claim.

5.1 Results and numerical validation

Table 1 records the actual frozen problem sizes. These are large coefficient vectors, even though only eight low-resolution views enter the main objective. The preparation stage took about eight minutes per scene; this cost is common to all color optimizers and is outside their reported seconds.

Table 1: Actual problem sizes. Geometry-training, test and appearance-validation columns count views. L bounds the unnormalized Hessian; η is the selected Adam rate.

Scene	Gaussians	Geom. train	Test	Val.	L	η
Train	334,056	263	38	24	3313	0.003
Truck	531,947	219	32	20	1283	0.003

The installed accelerated backward initially failed the Truck adjoint check (relative scalar discrepancy 0.036) and a directional derivative check (0.074). Varying the finite-difference step from 10^{-3} to 1 left the finite-difference derivative stable, so simply loosening a threshold was not justified.

We compiled the official Graphdeco reference rasterizer into a project-local directory and reran every appearance result below. The initial runs are preserved as diagnostic data and excluded from quantitative conclusions. Common geometry preparation still uses the original installation. The accepted reference source revision is saved with the code; this explicitly separates geometry preparation from the audited appearance operator.

Table 2: Relative operator-check errors for accepted main runs. The dense solver check also agrees with a direct solve to below 4×10^{-6} in solution norm.

Scene	Linearity	Adjoint	Symmetry	Derivative
Train	3.13e-07	1.64e-06	2.73e-06	8.21e-07
Truck	3.06e-07	3.27e-07	2.15e-06	6.64e-06

Table 3: Equal-budget main comparison at 100 updates, $\lambda = 0.01$. Objective and PSNR show mean $\pm$ sample standard deviation across three color perturbations; residual and time show means. F is scaled by 10^3. Times measure synchronized solver updates only.

Scene	Method	$10^3 F$	Residual ratio	Test PSNR	Seconds
Train	GD	4.763 ± 0.014	4.17e-01	19.13 ± 0.01	5.33
Train	Nesterov	2.750 ± 0.008	3.12e-01	17.95 ± 0.02	5.32
Train	Adam	1.913 ± 0.004	1.59e-01	18.49 ± 0.01	5.38
Train	CG	1.176 ± 0.005	8.56e-02	17.95 ± 0.02	5.59
Train	PCG	0.962 ± 0.000	3.45e-03	17.11 ± 0.00	5.56
Truck	GD	3.198 ± 0.010	4.45e-01	21.45 ± 0.01	6.45
Truck	Nesterov	1.753 ± 0.004	2.92e-01	20.99 ± 0.00	6.43
Truck	Adam	1.250 ± 0.002	1.23e-01	22.04 ± 0.00	6.46
Truck	CG	0.817 ± 0.002	1.08e-01	20.99 ± 0.02	6.57
Truck	PCG	0.757 ± 0.000	2.70e-03	20.74 ± 0.00	6.84

For Train, PCG reaches the lowest mean objective (0.000962), while GD has the highest test PSNR among the five (19.13 dB). The unperturbed preparation-color baseline is 19.37 dB; the seed-0 perturbed start is 19.02 dB.

For Truck, PCG reaches the lowest mean objective (0.000757), while Adam has the highest test PSNR among the five (22.04 dB). The unperturbed preparation-color baseline is 21.67 dB; the seed-0 perturbed start is 21.23 dB.

The large difference between maximum row-sum curvature and typical coefficients makes the safe scalar GD step conservative. Nesterov improves training fit relative to GD, while PCG exploits the unequal per-Gaussian observation scale. A lower fitted-view objective does not necessarily improve unseen images: the small-view, weak-ridge system can fit appearance inconsistencies and geometric errors by changing colors. Slow optimization can act as implicit early regularization, so image quality and numerical stationarity must be judged separately. Timing differences at similar update counts also include residual-replacement overhead; no hardware-independent speedup is claimed.

5.2 Regularization, coverage and resolution

Sensitivity runs repeat all five methods with seed 0 for ridge coefficients 0.1, 1 and 10, 32 fitted views, and width 512. Adam's small validation pilot is repeated for each configuration. These are one-initialization sensitivity measurements, not additional independent-scene statistics. Because the objective divides a fixed ridge sum by the number of image values, changing resolution or view count at fixed λ also changes the ridge-to-data balance. These ablations therefore measure the full specified configuration; they do not isolate image resolution alone.

For Train, PCG test PSNR at ridge values $0.01, 0.1, 1, 10$ is $17.11, 18.44, 19.12, 19.40$ dB. At 1,000 updates, the recomputed CG/PCG residual ratios are $7.56e-03$ and $5.66e-05$, respectively.

For Truck, PCG test PSNR at ridge values $0.01, 0.1, 1, 10$ is $20.74, 21.85, 22.06, 21.83$ dB. At 1,000 updates, the recomputed CG/PCG residual ratios are $1.20e-04$ and $1.95e-05$, respectively.

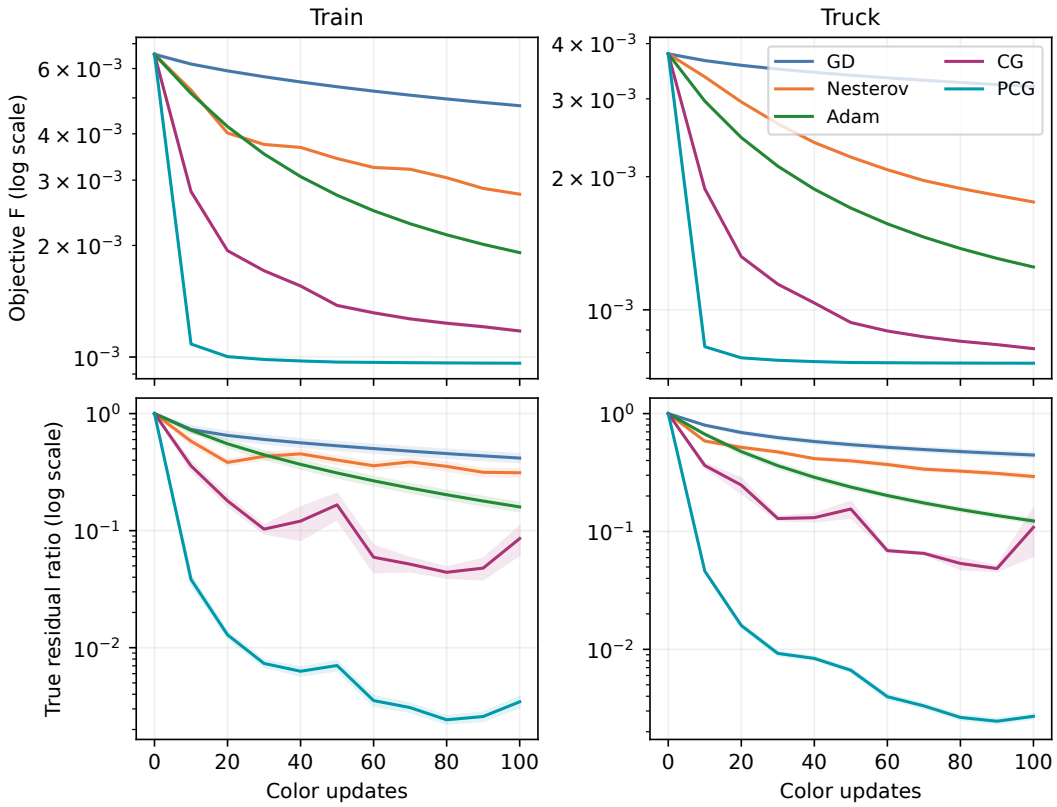


Figure 1: Main optimization histories. Curves average three matched color perturbations; shading shows one sample standard deviation. Residuals are recomputed from the true iterate. Objective and residual measure different aspects of convergence.

Table 4: Coverage/resolution sensitivity, seed 0 and $\lambda = 0.01$. Each method reports $10^3 F$, test raw PSNR (dB), and solver seconds. Objectives with different view sets are different problems.

		Adam			PCG		
Scene	Configuration	$10^3 F$	PSNR	Sec.	$10^3 F$	PSNR	Sec.
Train	8 views, width 256	1.909	18.49	5.41	0.962	17.11	5.54
Train	32 views, width 256	2.220	19.71	21.06	1.403	18.55	21.69
Train	8 views, width 512	2.367	18.25	4.58	1.334	16.14	4.79
Truck	8 views, width 256	1.251	22.04	6.34	0.757	20.74	6.86
Truck	32 views, width 256	1.217	23.63	28.16	1.096	23.26	18.27
Truck	8 views, width 512	0.991	22.60	7.44	0.663	20.69	7.80

Stronger ridge keeps unobserved or weakly observed colors closer to the preparation solution. Greater view coverage can reduce a fitting/generalization mismatch, but neither this observation nor the two-scene sample establishes a universal best regularizer. A larger-resolution fit additionally changes rasterization footprints and the number of observations. The full per-view raw/clipped PSNR and clipped SSIM records are included in the supplement, rather than silently replacing the optimized metric with a visually favorable one.

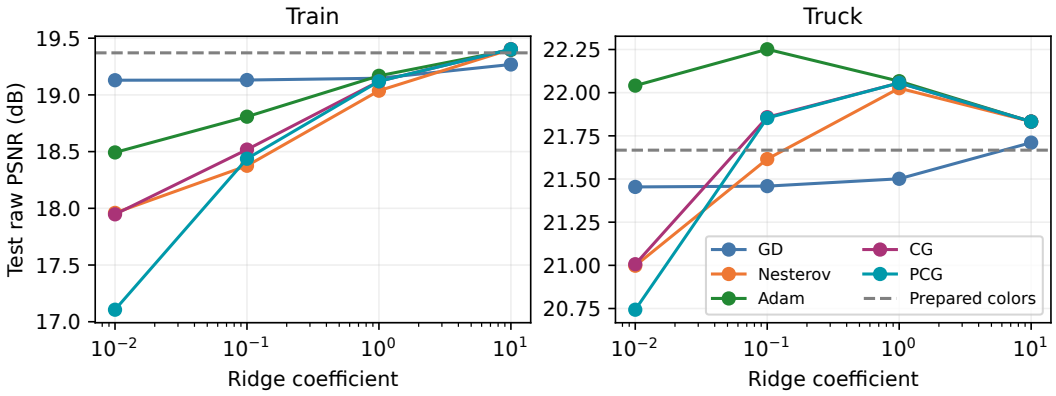


Figure 2: Test image quality versus ridge strength after 100 updates, seed 0. The dashed line is the shared unperturbed preparation-color baseline. These test curves are descriptive sensitivity results; test images did not choose optimizer learning rates.

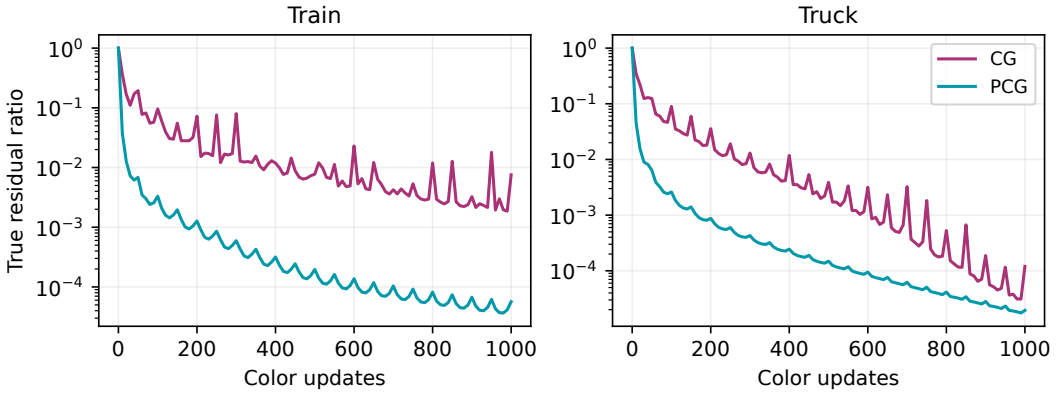


Figure 3: Longer CG/PCG stationarity checks with 1,000 updates and the main configuration, seed 0. Periodic residual replacement restarts the directions every 25 updates. These curves test finite-budget behavior rather than certify an exact optimum.



Figure 4: Same held-out view per scene, chosen as the middle name-sorted test image. Images are clipped for display. Each prediction is from seed 0 of the main weak-ridge configuration. Training-objective ranking is not sufficient to rank these held-out appearances.

6 Limitations and conclusions

Across the completed comparisons, the useful distinction is between solving the specified quadratic and choosing a model that generalizes. The mass-based preconditioner makes PCG a strong numerical baseline, while the sensitivity study shows why regularization and observation coverage belong in any interpretation of optimizer rankings. These are conditional empirical findings on two scenes, not a claim that one optimizer dominates full radiance-field training.

The model deliberately removes several important difficulties of full 3DGS. Geometry and opacity cannot adapt, a degree-zero color cannot represent specular view dependence, and an unclipped unconstrained iterate may contain out-of-range values. Ridge regularization trades data fit against preservation of preparation colors. Low-resolution, fitted-view subsets reduce cost but can change visibility footprints and generalization. Further work should study box-constrained colors, higher-degree linear spherical-harmonic models with carefully handled nonnegativity, larger view sets and alternating geometry/appearance updates. None of the present rates establish convergence for those broader nonlinear systems.

The reported first-order methods also represent specific choices: GD and Nesterov use a theorem-driven scalar bound, while Adam receives a small validation search. We do not exhaustively tune GD steps or momentum, vary geometry seeds, or claim a universally optimal optimizer ranking. Periodic CG restarts and float32 arithmetic further separate these measurements from ideal exact-arithmetic theory.

The central lesson is methodological: identify which scene variables determine visibility before choosing an optimizer. Once the renderer truly becomes linear in the chosen coefficients, numerical linear algebra provides interpretable baselines and verifiable assumptions. A lower training objective must still be evaluated separately from held-out image quality.

Reproducibility and code

The companion archive `COMP6704_3DGS_Appearance_Reproducibility.zip` contains the optimizer source, documented commands, environment export, source hashes, split and input manifests, raw histories, evaluation JSON files, figure-generation script and this report's LaTeX content. The complete independent server project additionally retains the copied renderer source and geometry/color checkpoints. It resides at `COMP6704_individual_appearance_20261027` under the authorized server's `data/3D_Research` directory. No login credentials are included. The installed renderer has local modifications and no available Git commit identifier, so a file hash manifest is used instead of inventing an upstream revision. Input data must be supplied separately. No public code URL is asserted, because this work has not been published to a repository.

References

S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. `https://web.stanford.edu/~boyd/cvxbook/`.

M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952. `https://nvlpubs.nist.gov/nistpubs/jres/049/jresv49n6p409_A1b.pdf`.

L. Höllein, A. Božič, M. Zollhöfer and M. Nießner. 3DGS-LM: Faster Gaussian-splatting optimization with Levenberg–Marquardt. *ICCV*, 2025. `https://arxiv.org/abs/2409.12892`.

R. Hsiao, Y. Fang, X. Huang, R. Li, H. Rabeti, Z. Gojcic, J. Lavaei, J. Demmel and S. Shao. 3DGS^2-TR: Scalable second-order trust-region method for 3D Gaussian Splatting. arXiv:2602.00395, 2026. `https://arxiv.org/abs/2602.00395`.

B. Kerbl, G. Kopanas, T. Leimkühler and G. Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. doi:10.1145/3592433. `https://arxiv.org/abs/2308.04079`.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *ICLR*, 2015. `https://arxiv.org/abs/1412.6980`.

A. Knapitsch, J. Park, Q.-Y. Zhou and V. Koltun. Tanks and Temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4), 2017. https://www.tanksandtemples.org/.

Y. E. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269(3):543–547, 1983. English translation: https://hengshuaiyao.github.io/papers/nesterov83.pdf.

H. Pehlivan, A. Boscolo Camiletto, L. G. Foo, M. Habermann and C. Theobalt. Matrix-free second-order optimization of Gaussian splats with residual sampling. arXiv:2504.12905v3, 2026. https://arxiv.org/abs/2504.12905v3.

S. J. Reddi, S. Kale and S. Kumar. On the convergence of Adam and beyond. *ICLR*, 2018. https://openreview.net/forum?id=ryQu7f-RZ.